CCSDS TM Framer: Structure, Segmentation, and Bulk Payload Handling

(1115-byte frames; RS(255,223) with Interleave $I = 5$; ASM+Conv Chain)

Joshua Wille

October 29, 2025

Contents

1 Purpose and Scope

This note specifies a practical Transfer Frame (TM) framer for a classic CCSDS Telemetry downlink using **1115-byte** TM frames mapped to **RS(255,223)** with interleave depth $I = 5$, followed by PRBS randomization of the codeblock, insertion of the 32-bit Attached Sync Marker (ASM), and convolutional encoding ($r = 1/2$, $K = 7$). It focuses on: (i) exact frame sizing, (ii) the First Header Pointer (FHP), (iii) packet segmentation across frames, and (iv) handling bulk payloads (Space Packets > 1115 B).

2 Where the TM Framer Lives in the Chain

TX: SPP → **TM Framer (1115 B)** → RS(255,223) with $I = 5$ (1275 B) → PRBS8 randomize (codeblock scope) → prepend ASM (1279 B) → Conv. encoder ($r = 1/2$, $K = 7$) → modulation.

RX: demod/Viterbi → find/strip ASM → derandomize RS codeblock → deinterleave + RS decode → TM (1115 B) → **deframe SPP using FHP**.

3 Transfer Frame Structure (1115-byte total)

Let $L_{\text{hdr}} = 6\,\text{B}$ and $L_{\text{body}} = 1109\,\text{B}$. One TM frame is

$$L_{\text{TM}} = L_{\text{hdr}} + L_{\text{body}} = 6 + 1109 = \mathbf{1115\,B}. \tag{1}$$

The body (*packet zone*) holds Space Packet bytes (possibly segmented). The primary header contains, at minimum for this implementation note, a frame counter and the **First Header Pointer (FHP)**.

Why 1115 B is *nice* with RS$(255, 223)$, $I = 5$

Five interleave rows of 223 information bytes yield exactly $5 \times 223 = \mathbf{1115}\,\text{B}$ per RS codeblock input. After coding, $1115 + 5 \times 32 = 1275\,\text{B}$. Appending the 4 B ASM gives 1279 B before convolutional encoding.

4 First Header Pointer (FHP): Definition and Rules

The **FHP** is an integer in the range [0, 2047] placed in the TM primary header to indicate the location of the *first new Space Packet Primary Header* within the 1109-byte data field.

- If the frame **begins at a packet boundary**, then FHP = 0.
- If the frame **starts with a continuation** (i.e., the first bytes belong to the middle of a previously-started Space Packet), then FHP is the *byte offset* (0–1108) where the first *new* Space Packet header begins inside this frame.
- If *no new header appears* anywhere in the 1109-byte data field (only continuation/fill), set FHP to the standard's *"no header present"* value (denoted here as $\text{FHP}_{\varnothing}$).

5 Segmentation Policy (Space Packets → TM Frames)

Goals

1. Maximize utilization of the 1109-byte packet zone with minimal fill.
2. Maintain Space Packet integrity: segmentation is byte-accurate, with SPP sequence flags reflecting *first/continuation/last/unsegmented*.
3. Compute FHP consistently from the *written* data field.

Packing Algorithm (Deterministic Recipe)

Given an input queue of SPPs (each provided with a known length):

Step 1: If a *partial packet* is in progress from the prior frame, continue writing its remaining bytes first.

Step 2: Otherwise, while space remains in the 1109-byte zone:

a) If the next whole SPP fits, write it entirely.

b) Else, start a *segment*: write as many bytes as fit and carry the remainder to the next frame.

Step 3: Record the offset where the *first new SPP header* enters this frame (this becomes FHP). If no new header appears, set FHP = $\text{FHP}_{\varnothing}$.

Step 4: If the queue empties before filling the frame, pad the remainder with fill (e.g., 0x00).

Notes. The framer should emit frames either (i) when the packet zone is full, or (ii) when a latency/timeout occurs to prevent head-of-line blocking. With segmentation, frames rarely carry large fill runs.

Pseudocode

```
state: queue<SPP>, cur_pkt=None, cur_off=0
function build_frame():
  body = byte[1109]
  w = 0
  first_new = NONE

  # continue partial packet first
  if cur_pkt != None and cur_off > 0:
    n = min(len(cur_pkt)-cur_off, 1109-w)
    copy cur_pkt[cur_off:cur_off+n] -> body[w:w+n]
    w += n; cur_off += n
    if cur_off == len(cur_pkt): cur_pkt=None; cur_off=0

  while w < 1109:
    if cur_pkt == None:
      if queue.empty(): break
      cur_pkt = queue.pop_front(); cur_off = 0
      if first_new is NONE: first_new = w # new SPP header starts here

    n = min(len(cur_pkt)-cur_off, 1109-w)
    copy cur_pkt[cur_off:cur_off+n] -> body[w:w+n]
    w += n; cur_off += n

    if cur_off == len(cur_pkt):
      cur_pkt=None; cur_off=0 # whole packet fit
    else:
      break # ran out of space mid-packet; continue next frame

  if w < 1109: pad body[w:1109] with 0x00
  FHP = first_new if first_new is not NONE else FHP_NONE
  return body, FHP
```

6 Bulk Payloads (Space Packets > 1115 B)

Bulk payloads exceeding one frame are handled naturally by segmentation:

- **One large Space Packet:** the framer writes a start segment in Frame N, continuation segments in subsequent frames, and a last segment in the final frame. The Space Packet's own *Sequence Flags* encode the segmentation type; the Space Packet Sequence *Count* advances per packet, not per segment.
- **Many smaller packets:** pack as many whole SPPs as fit; when one more will not fit, write its head segment and complete it at the beginning of the next frame. This minimizes fill and latency.

7 Randomizer, ASM, and Coding Placement

1. For RS+Conv chains, apply the PRBS8 randomizer over the *RS-interleaved codeblock* (do not include the ASM).
2. Insert the **ASM** (32-bit sync marker) *after* randomization and *before* convolutional encoding.
3. Leave the ASM unrandomized to preserve correlator performance at the receiver.

8 Validation Checklist

1. Input SPPs arrive as a tagged stream with per-packet length.
2. Output frames are exactly 1115 B, one length-tag per frame.
3. FHP equals: 0 when data-byte 0 starts a new SPP; an in-frame offset to the first new SPP header; or the "no header" value when the frame carries only continuation/fill.
4. With segmentation enabled, utilization of the 1109-byte packet zone is $\gg$ when compared to one-SPP-per-frame; long 0x00 pads largely disappear.
5. Downstream lengths align: 1115 B $\rightarrow$ 1275 B (after RS) $\rightarrow$ 1279 B (after ASM) $\rightarrow$ post-conv length for the selected code rate.

9 Engineering Tips

- Use an emission policy: emit as soon as (i) the body is full, or (ii) a latency timer expires; otherwise you risk periodic underfilled frames during low traffic.
- Keep a small frame counter in the primary header for bench analysis; it helps correlate dumps and scope captures.
- Place the randomizer and ASM exactly as above; never randomize the ASM.

10 Symbols and Sizes (Quick Reference)

Quantity	Value
TM header L_{hdr}	6 B
TM body L_{body}	1109 B
TM frame L_{TM}	1115 B
RS code (N, K)	$(255, 223)$
Interleave depth I	5 (rows)
RS parity per row	32 B (i.e., $255 - 223$)
Post-RS size	$1115 + 5 \times 32 = 1275$ B
ASM	4 B (unrandomized)
Post-ASM size	1279 B
Conv. code	$r = 1/2,\ K = 7$ (171/133)

11 References

- CCSDS 131.0-B-3 (Blue Book): *TM Synchronization and Channel Coding.* (Draft ECSS-aligned copy provided in project materials.)

- ECSS-E-ST-50-01C (31 July 2008): *Space Engineering — Communications.* (TM/AOS framing, randomization and placement guidance.)
- Project staging notes / sizing sheets: mapping 1115 B TM frames $\rightarrow$ RS$\times I = 5$, ASM placement, and post-convolution lengths used in the RSCL lab chain.